

MANUAL TÉCNICO

PORTADA

Título	Manual Técnico — SmartStore
Subtítulo	Especificación técnica de arquitectura, implementación y despliegue
Referencia	SS-MT-001
Versión	1.0.0
Estado	Publicado
Fecha de emisión	18 de mayo de 2026
Fecha de revisión	18 de mayo de 2027
Autor	Equipo de Desarrollo SmartStore
Revisado por	Ingeniería del Conocimiento — Facultad de Ingeniería, UNAM
Institución	Universidad Nacional Autónoma de México
Clasificación	Técnico / Restringido a desarrolladores
Idioma	Español (México)

HISTORIAL DE REVISIONES

0.1	01-may-2026	Equipo SmartStore	Estructura inicial, modelos de datos
0.5	10-may-2026	Equipo SmartStore	API REST, ontología, búsqueda semántica
0.9	16-may-2026	Equipo SmartStore	Deployment, seguridad, testing
1.0.0	18-may-2026	Equipo SmartStore	Versión publicada

NORMAS DE REFERENCIA

IEEE 830-1998	Software Requirements Specification
ISO/IEC/IEEE 29148:2018	Systems and software engineering — Life cycle processes — Requirements engineering
ISO/IEC 27001:2022	Information security management systems
ISO/IEC 25010:2011	Software product quality model
W3C OWL 2.0 (2012)	OWL 2 Web Ontology Language — Document Overview

W3C SPARQL 1.1 (2013)

SPARQL 1.1 Query Language

W3C RDF 1.1 (2014)

Resource Description Framework — Concepts

RFC 7519 (2015)

JSON Web Token (JWT)

RFC 7617 (2015)

The ‘Basic’ HTTP Authentication Scheme

TABLA DE CONTENIDOS

- [Cap. 1 — Introducción técnica](#)
- [Cap. 2 — Arquitectura del sistema](#)
- [Cap. 3 — Stack tecnológico](#)
- [Cap. 4 — Base de datos](#)
- [Cap. 5 — API REST](#)
- [Cap. 6 — Frontend — Arquitectura React](#)
- [Cap. 7 — Backend — Arquitectura Django](#)
- [Cap. 8 — Ontología OWL/SPARQL](#)
- [Cap. 9 — Búsqueda semántica](#)
- [Cap. 10 — Seguridad](#)
- [Cap. 11 — Rendimiento y optimización](#)
- [Cap. 12 — Deployment](#)
- [Cap. 13 — Testing](#)
- [Cap. 14 — Troubleshooting técnico](#)
- [Cap. 15 — Glosario técnico](#)
- [Apéndice A — ERD completo](#)
- [Apéndice B — Diagrama de arquitectura](#)
- [Apéndice C — Queries SPARQL de referencia](#)
- [Apéndice D — Variables de entorno](#)
- [Apéndice E — Scripts útiles](#)
- [Apéndice F — Referencias](#)

CAPÍTULO 1 — INTRODUCCIÓN TÉCNICA

1.1 Propósito del documento

Este Manual Técnico especifica la arquitectura de software, los modelos de datos, los contratos de API, los mecanismos de búsqueda semántica y los procedimientos de despliegue del sistema SmartStore. Sirve como referencia autorizada para desarrolladores, arquitectos de software y administradores de sistemas involucrados en el ciclo de vida del proyecto.

El documento cumple con los requisitos de documentación técnica establecidos por ISO/IEC/IEEE 29148:2018 e incorpora principios de seguridad definidos en ISO/IEC 27001:2022.

1.2 Público objetivo

Desarrollador Backend

Cap. 4, 5, 7, 8, 9, 10

Desarrollador Frontend	Cap. 5, 6
Arquitecto de software	Cap. 2, 3, 8, 11
DevOps / SRE	Cap. 12, 13, 14
Analista de seguridad	Cap. 10, Apéndice D
Investigador en IA semántica	Cap. 8, 9, Apéndice C

1.3 Convenciones técnicas

<code>monoespaciado</code>	Código, ruta de archivo, comando de terminal, nombre de variable
Negrita	Término técnico definido en el glosario (primera aparición)
<code>GET /api/ruta/</code>	Endpoint HTTP con método explícito
<code>→</code>	Flujo de datos o redirección
<code>[REQUERIDO]</code>	Campo o parámetro obligatorio
<code>[OPCIONAL]</code>	Campo o parámetro no obligatorio
<code>❗</code>	Advertencia de seguridad o comportamiento crítico

1.4 Acrónimos y abreviaturas

API	Application Programming Interface
ASGI	Asynchronous Server Gateway Interface
CORS	Cross-Origin Resource Sharing
CSRF	Cross-Site Request Forgery
DRF	Django REST Framework
ERD	Entity-Relationship Diagram
FK	Foreign Key (clave foránea)
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
IRI	Internationalized Resource Identifier
JSON	JavaScript Object Notation
JWT	JSON Web Token (RFC 7519)
ORM	Object-Relational Mapper
OWL	Web Ontology Language (W3C)
PK	Primary Key (clave primaria)
RDF	Resource Description Framework (W3C)
RDFS	RDF Schema
REST	Representational State Transfer
SPARQL	SPARQL Protocol and RDF Query Language (W3C)

SPA	Single Page Application
SQL	Structured Query Language
TLS	Transport Layer Security
URI	Uniform Resource Identifier
WSGI	Web Server Gateway Interface
XSS	Cross-Site Scripting

CAPÍTULO 2 — ARQUITECTURA DEL SISTEMA

2.1 Visión general — Arquitectura de 3 capas

SmartStore implementa una arquitectura cliente-servidor de tres capas con separación estricta de responsabilidades:



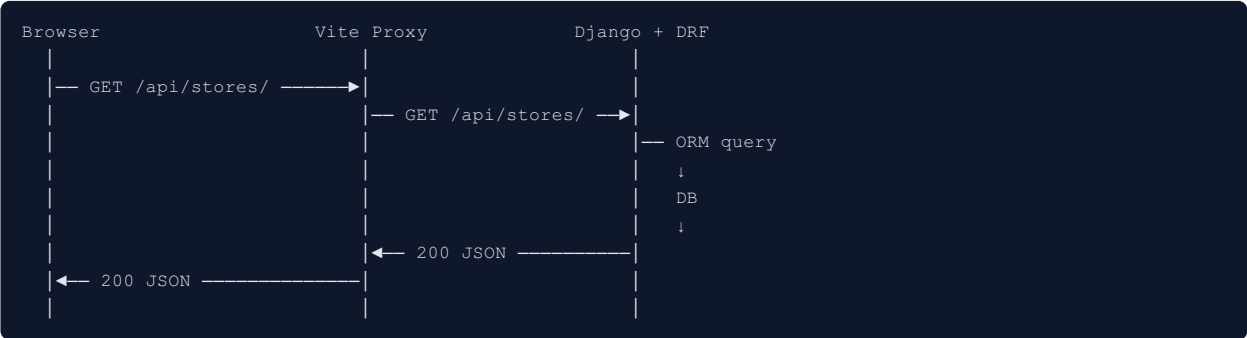
2.2 Componentes del frontend

Router	React Router v6	Navegación SPA, rutas protegidas
Pages	React 18 (JSX)	Vistas completas por ruta
UI Components	React + Tailwind v4	Elementos reutilizables
Context API	React Context	Estado global (auth, carrito, toasts)
Data fetching	TanStack Query v5	Cache, sincronización, loading states
HTTP Client	Axios 1.7	Peticiones HTTP con interceptors JWT
Icons	Lucide React 0.468	Iconografía vectorial SVG
Styling	Tailwind CSS v4	Utility-first CSS con CSS variables
Build tool	Vite 6.0	Bundling, HMR, dev server con proxy

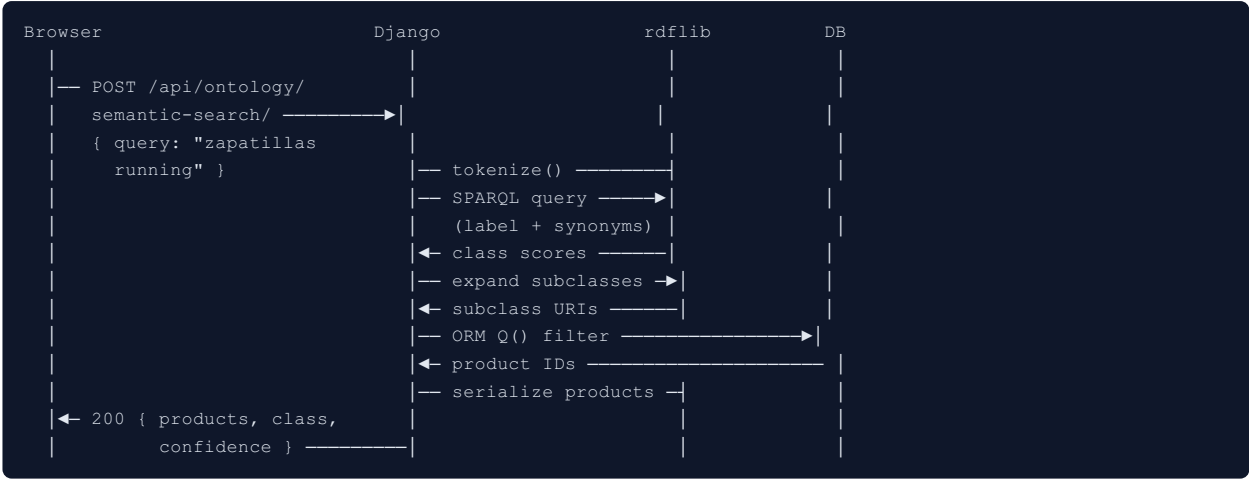
2.3 Componentes del backend

Web framework	Django 5.1	Routing, ORM, admin, middleware
REST layer	DRF 3.15	Serialización, ViewSets, permisos
Auth	SimpleJWT 5.3	Emisión y validación de JWT
CORS	django-cors-headers 4.6	Cabeceras CORS para el SPA
Filters	django-filter 24.3	Filtrado declarativo de querysets
Ontology engine	rdflib 7.1	Parsing OWL, grafo RDF, consultas SPARQL
SPARQL remote	SPARQLWrapper 2.0	Consultas a endpoints SPARQL externos
Image handling	Pillow 11.0	Procesamiento de avatares
Config	python-decouple 3.8	Variables de entorno desde .env
DB driver	psycopg2-binary 2.9	Adaptador PostgreSQL

2.4 Flujo de datos — Petición REST típica



2.5 Flujo de datos — Búsqueda semántica



2.6 Comunicación entre capas

Browser → Vite proxy	HTTP/1.1	—	—
Vite proxy → Django	HTTP/1.1	JSON	Bearer JWT
Django → PostgreSQL	TCP (psycopg2)	SQL binary	DB credentials
Django → OWL file	File I/O	RDF/XML	—
Django → rdflib	In-process	Python objects	—

CAPÍTULO 3 — STACK TECNOLÓGICO

3.1 Frontend

Framework UI	React	18.3.1	Renderizado declarativo, Virtual DOM
DOM renderer	React DOM	18.3.1	Montaje en el navegador
Router	React Router DOM	6.28.0	SPA client-side routing, rutas anidadas
HTTP client	Axios	1.7.9	Peticiones HTTP, interceptors JWT
Server state	TanStack Query	5.62.0	Cache, deduplicación, background sync
Icons	Lucide React	0.468.0	SVG icons tree-shakeable
CSS utility	clsx	2.1.1	Composición condicional de clases
CSS merge	tailwind-merge	2.5.5	Merge de clases Tailwind sin conflictos
Estilos	Tailwind CSS	4.1.0	Utility-first, CSS variables via @theme
Build tool	Vite	6.0.5	ESM-native bundler, HMR, proxy
React plugin	@vitejs/plugin-react	4.3.4	JSX transform, Fast Refresh
Tailwind plugin	@tailwindcss/vite	4.1.0	Integración Tailwind v4 con Vite

3.2 Backend

Web framework	Django	5.1.4	MVC, ORM, admin, middleware
REST framework	Django REST Framework	3.15.2	Serializers, ViewSets, parsers
JWT auth	djangorestframework-simplejwt	5.3.1	Emisión, validación, blacklist de tokens
CORS	django-cors-headers	4.6.0	Cabeceras CORS para el SPA
Filtering	django-filter	24.3	FilterSets declarativos
DB driver	psycopg2-binary	2.9.10	Adaptador PostgreSQL (C extension)
Config	python-decouple	3.8	Variables de entorno desde <code>.env</code>
Images	Pillow	11.0.0	Procesamiento de imágenes (avatares)
Ontology	rdflib	7.1.1	RDF/OWL parsing, SPARQL 1.1 in-memory
SPARQL remote	SPARQLWrapper	2.0.0	Ciente para endpoints SPARQL externos

3.3 Base de datos

Desarrollo	SQLite	3.x (built-in Python)	Activado con <code>USE_SQLITE=True</code>
Producción	PostgreSQL	15+	Default; requiere <code>psycopg2</code>

3.4 Ontología / Web Semántica

OWL 2	W3C 2012	Formato del archivo <code>smartstore.owl</code>
RDF 1.1	W3C 2014	Serialización RDF/XML del grafo
RDFS	W3C 2014	<code>rdfs:label</code> , <code>rdfs:subClassOf</code> , <code>rdfs:comment</code>
SPARQL 1.1	W3C 2013	Consultas al grafo en memoria (rdflib)
Namespace	—	<code>http://www.smartstore.org/ontology#</code>

3.5 Infraestructura (producción recomendada)

WSGI server	Gunicorn	Servidor de aplicación Django
Reverse proxy	Nginx	Proxy, TLS, servir archivos estáticos
Process manager	systemd	Supervisión y reinicio de Gunicorn
Frontend host	S3 + CloudFront	Archivos estáticos del SPA
DB	PostgreSQL 15	Base de datos relacional
SSL/TLS	Let's Encrypt	Certificados HTTPS gratuitos

CAPÍTULO 4 — BASE DE DATOS

4.1 Diagrama Entidad-Relación (ERD)




```

| índices: product_store_active_idx(store, is_active)
|          product_price_idx(price)
|          product_rating_idx(rating)
|          product_store_price_idx(store, price)
|          product_ontology_active_idx(ontology_class, is_active)

```

```

| 1 (SET_NULL on delete)
| FK nullable
| N

```

```

| orders_orderitem

```

```

| PK id          BigAutoField
| FK order_id    → orders_order.id  CASCADE
| FK product_id  → products_product.id SET_NULL NULL
| name          VARCHAR(300) (snapshot del nombre)
| quantity      PositiveInt
| price_at_purchase DECIMAL(12,2) (snapshot del precio)
| subtotal      DECIMAL(12,2)

```

```

| N
| FK CASCADE
| 1

```

```

| orders_order

```

```

| PK id          BigAutoField
| FK user_id     → auth_user.id    CASCADE
| order_number   VARCHAR(20)  UNIQUE auto (UUID hex 8 chars)
| subtotal      DECIMAL(12,2)
| tax           DECIMAL(12,2)
| total         DECIMAL(12,2)
| status        VARCHAR(20)  DEFAULT 'pending'
|              choices: pending|confirmed|shipped|
|                  delivered|completed|cancelled
| created_at    TIMESTAMPTZ  auto_now_add
| updated_at    TIMESTAMPTZ  auto_now

```

4.2 Decisiones de diseño del modelo de datos

OrderItem.product es nullable (SET_NULL): Si un producto se elimina de la base de datos, el historial de compras conserva el registro del ítem con `name` y `price_at_purchase` intactos. El campo `product_id` quedará `NULL` pero la información comercial persiste.

Product.semantic_tags como JSONField: Almacena una lista de strings como `["5G", "Bluetooth 5.0", "IP68"]`. Esto evita una tabla intermedia N:M para atributos que varían por categoría. El filtrado se realiza con `json_each()` en SQLite y operador `__contains` en PostgreSQL.

Store.categories como JSONField: Lista plana de strings (`["Electrónica", "Audio"]`). No se usa una tabla de categorías separada por YAGNI; la normalización se deja para versiones futuras.

Order.order_number auto-generado: Se genera en `save()` con `uuid4().hex[:8].upper()`. Se retentará hasta 10 veces ante colisión de `UNIQUE` constraint.

4.3 Máquina de estados — Order.status

Implementado en `OrderStatusUpdateSerializer.validate_status()` mediante el diccionario `ALLOWED_STATUS_TRANSITIONS`:

4.4 Migraciones Django

users	0001_initial	Crea users_userprofile
stores	0001_initial	Crea stores_store con ImageField
stores	0002_store_logo_banner_urlfield	Convierte logo/banner a URLField
products	0001_initial	Crea products_product con ImageField
products	0002_product_image_urlfield	Convierte image a URLField
orders	0001_initial	Crea orders_order y orders_orderitem
orders	0002_add_completed_status	Añade estado completed a choices

Ejecutar todas las migraciones:

```
python manage.py migrate
```

Verificar el estado de las migraciones:

```
python manage.py showmigrations
```

CAPÍTULO 5 — API REST

5.1 Convenciones de la API

Base URL

http://localhost:8000 (desarrollo)

Formato	JSON exclusivamente (Content-Type: application/json)
Autenticación	Bearer JWT en cabecera Authorization
Paginación	PageNumberPagination, PAGE_SIZE = 20 (desactivada en orders)
Versionamiento	No implementado (v1 implícita en prefijo /api/)
Errores	{ "detail": "mensaje" } 0 { "campo": ["error"] }

5.2 Autenticación JWT

FLUJO COMPLETO

```

1. POST /api/auth/login/
   Body: { email, password }
   Response: { access, refresh, user }

2. Almacenamiento cliente:
   localStorage.setItem('ss_access', access)
   localStorage.setItem('ss_refresh', refresh)

3. Peticiones autenticadas:
   Authorization: Bearer <access_token>

4. Renovación (access expirado - 60 min):
   POST /api/auth/token/refresh/
   Body: { refresh }
   Response: { access, refresh } ← nuevo par (ROTATE_REFRESH_TOKENS=True)

5. Logout (blacklist del refresh token):
   POST /api/auth/logout/
   Body: { refresh }
   → RefreshToken(refresh).blacklist()

```

CONFIGURACIÓN JWT (SETTINGS.PY)

```

SIMPLE_JWT = {
    'ACCESS_TOKEN_LIFETIME': timedelta(minutes=60),
    'REFRESH_TOKEN_LIFETIME': timedelta(days=7),
    'ROTATE_REFRESH_TOKENS': True, # nuevo refresh en cada refresh
    'BLACKLIST_AFTER_ROTATION': True, # invalida el refresh anterior
    'AUTH_HEADER_TYPES': ('Bearer',),
}

```

5.3 Tabla completa de endpoints

5.3.1 AUTENTICACIÓN (/API/AUTH/)

POST	/api/auth/register/	No	Crear cuenta nueva
POST	/api/auth/login/	No	Obtener access + refresh
POST	/api/auth/logout/	No*	Blacklistear refresh token
GET	/api/auth/me/	Sí	Datos del usuario autenticado
PUT	/api/auth/profile/	Sí	Actualizar perfil completo
POST	/api/auth/token/refresh/	No	Renovar access token

*El logout acepta tokens anónimos y trata el error silenciosamente.

POST /api/auth/register/ — Request:

```
{
  "full_name": "Ana García",
  "email": "ana@email.com",
  "password": "password123",
  "password2": "password123",
  "phone": "+52 55 1234 5678",
  "city": "Ciudad de México"
}
```

POST /api/auth/register/ — Response 201:

```
{
  "access": "<jwt_access_token>",
  "refresh": "<jwt_refresh_token>",
  "user": {
    "id": 1,
    "email": "ana@email.com",
    "full_name": "Ana García",
    "profile": {
      "phone": "+52 55 1234 5678",
      "city": "Ciudad de México",
      "address": "",
      "avatar": null
    }
  }
}
```

POST /api/auth/login/ — Request:

```
{ "email": "ana@email.com", "password": "password123" }
```

Response 200: Mismo formato que register.

Response 401: {"detail": "No active account found with the given credentials"}

5.3.2 TIENDAS (/API/STORES/)

GET	/api/stores/	No	search, ordering
GET	/api/stores/{slug}/	No	—
GET	/api/stores/{slug}/products/	No	Ver tabla 5.3.3

GET /api/stores/ — Response 200 (fragmento):

```
{
  {
    "id": 1,
    "name": "Nike Store",
    "slug": "nike-store",
    "description": "...",
    "logo": "https://images.unsplash.com/...",
    "banner": "https://images.unsplash.com/...",
    "categories": ["Calzado", "Ropa Deportiva"],
    "product_count": 20,
    "is_active": true
  }
}
```

5.3.3 PRODUCTOS (/API/PRODUCTS/ Y /API/STORES/{SLUG}/PRODUCTS/)

price_min	number	Precio mínimo (gte)
price_max	number	Precio máximo (lte)
rating_min	number	Calificación mínima (gte)
has_discount	boolean	Solo con descuento activo
ontology_class	string	Filtro por clase OWL (icontains)
search	string	Búsqueda en nombre y descripción
ordering	string	price, -price, rating, -rating, -created_at
<attr>	string	Filtro dinámico de semantic_tags (ej: ?hasSurface=Trail)

GET /api/products/{id}/ — Response 200:

```
{
  "id": 42,
  "name": "Nike Air Max 270",
  "slug": "nike-air-max-270",
  "description": "...",
  "price": "2499.00",
  "compare_price": "2999.00",
  "discount_percent": 17,
  "image": "https://images.unsplash.com/...",
  "rating": "4.80",
  "rating_count": 1234,
  "semantic_tags": ["Running", "Amortiguación máxima", "Sintético"],
  "ontology_class": "http://www.smartstore.org/ontology#RunningShoe",
  "store": { "id": 1, "name": "Nike Store", "slug": "nike-store" }
}
```

5.3.4 ÓRDENES (/API/AUTH/ORDERS/)

GET	/api/auth/orders/	Sí	Historial del usuario (sin paginación)
POST	/api/auth/orders/	Sí	Crear nueva orden
PATCH	/api/auth/orders/{id}/	Sí	Actualizar status

POST /api/auth/orders/ — Request:

```
{
  "subtotal": 4998.00,
  "tax": 499.80,
  "total": 5497.80,
  "items": [
    {
      "product_id": 42,
      "name": "Nike Air Max 270",
      "quantity": 2,
      "price_at_purchase": 2499.00,
      "subtotal": 4998.00
    }
  ]
}
```

Response 201:

```
{
  "id": 7,
  "order_number": "A1B2C3D4",
  "subtotal": "4998.00",
  "tax": "499.80",
  "total": "5497.80",
  "status": "pending",
  "created_at": "2026-05-18T18:40:00Z",
  "items": [...]
}
```

PATCH /api/auth/orders/{id}/ — Request:

```
{ "status": "completed" }
```

Response 400 (transición inválida):

```
{ "status": ["Cannot transition from 'completed' to 'pending'."] }
```

5.3.5 ONTOLOGÍA (/API/ONTOLOGY/)

GET	/api/ontology/suggestions/?q=texto	No	Autocompletado (stores + products)
GET	/api/ontology/classes/	No	Todas las clases OWL
GET	/api/ontology/filters/?class=Smartphone	No	Filtros dinámicos por clase
POST	/api/ontology/semantic-search/	No	Búsqueda semántica
POST	/api/ontology/infer/	No	Inferir tags de un producto
POST	/api/ontology/reload/	No*	Recargar grafo (DEBUG=True solo)

POST /api/ontology/semantic-search/ — Request:

```
{ "query": "zapatillas running ligeras", "store_id": 1 }
```

Response 200:

```
{
  "query": "zapatillas running ligeras",
  "inferred_class": "RunningShoe",
  "inferred_class_uri": "http://www.smartstore.org/ontology#RunningShoe",
  "confidence": 0.92,
  "matched_tokens": ["zapatillas", "running", "ligeras"],
  "expanded_classes": ["RunningShoe", "SportswearProduct"],
  "count": 5,
  "products": [ ... ]
}
```

GET /api/ontology/filters/?class=Smartphone — Response 200:

```
{
  "class": "Smartphone",
  "filters": {
    "hasBrand": { "label": "Marca", "type": "select", "values": ["Samsung", "Apple", "Xiaomi"] },
    "hasRAM": { "label": "RAM", "type": "select", "values": ["4GB", "8GB", "12GB", "16GB"] },
    "hasStorage": { "label": "Almacenamiento", "type": "select", "values": ["128GB", "256GB"] },
    "hasScreenSize": { "label": "Pantalla", "type": "range", "values": null }
  },
  "filter_count": 4
}
```

5.4 Códigos de respuesta HTTP utilizados

200 OK	Éxito	GET, PATCH exitosos
201 Created	Recurso creado	POST register, POST order
204 No Content	Éxito sin cuerpo	Logout exitoso
400 Bad Request	Validación fallida	Campos inválidos, transición de status
401 Unauthorized	Token ausente o inválido	Endpoint protegido sin token válido
403 Forbidden	Sin permiso	reload_ontology en DEBUG=False
404 Not Found	Recurso no encontrado	Slug/ID inexistente
500 Internal Server Error	Error del servidor	Excepción no controlada

5.5 CORS

```
CORS_ALLOWED_ORIGINS = [
    'http://localhost:5173',
    'http://127.0.0.1:5173',
]
```

En producción se debe añadir el dominio del frontend desplegado. El middleware `CorsMiddleware` debe estar **antes** de `CommonMiddleware` en `MIDDLEWARE`.

CAPÍTULO 6 — FRONTEND — ARQUITECTURA REACT

6.1 Estructura de directorios

```

frontend/
├─ index.html           ← Entry point HTML
├─ package.json
├─ vite.config.js       ← Proxy /api → localhost:8000
├─ src/
│  ├─ main.jsx          ← Montaje React, providers globales
│  ├─ App.jsx           ← Router + rutas
│  ├─ api/
│  │  ├─ axiosConfig.js ← Instancia Axios + interceptors JWT
│  │  ├─ auth.js        ← authApi (login, register, profile, orders)
│  │  ├─ stores.js      ← getStores(), getStore(), getStoreProducts()
│  │  ├─ products.js    ← getProducts(), getProduct()
│  │  └─ ontology.js    ← getOntologyFilters(), semanticSearch()
│  ├─ context/
│  │  ├─ AuthContext.jsx ← user, login, logout, register, updateProfile
│  │  ├─ CartContext.jsx ← items, count, subtotal, addToCart, removeFromCart
│  │  ├─ ToastContext.jsx ← toast(msg, {type}) - notificaciones emergentes
│  │  └─ ThemeContext.jsx ← Detección y aplicación de dark mode
│  ├─ components/
│  │  ├─ layout/
│  │  │  ├─ Header.jsx  ← Nav, búsqueda semántica, carrito, menú usuario
│  │  │  └─ Footer.jsx  ← Pie de página
│  │  └─ ui/
│  │     ├─ ProductCard.jsx ← Tarjeta de producto con fallback de imagen
│  │     ├─ StoreCard.jsx  ← Tarjeta de tienda con banner + logo
│  │     ├─ FilterPanel.jsx ← Panel de filtros dinámicos
│  │     ├─ Breadcrumb.jsx ← Migas de pan
│  │     └─ LoadingGrid.jsx ← Skeletons de carga
│  └─ pages/
│     ├─ StoresCatalog.jsx ← / - Catálogo de tiendas
│     ├─ StorePage.jsx     ← /tienda/:slug
│     ├─ ProductDetailPage.jsx ← /producto/:slug
│     ├─ CartPage.jsx      ← /carrito
│     ├─ CheckoutPage.jsx  ← /checkout
│     ├─ LoginPage.jsx     ← /login
│     ├─ RegisterPage.jsx  ← /registro
│     └─ ProfilePage.jsx   ← /perfil
└─ lib/
   └─ utils.js            ← formatPrice(), cn() (clsx + tailwind-merge)

```

6.2 Configuración del proxy (Vite)

```

// vite.config.js
server: {
  proxy: {
    '/api': {
      target: 'http://localhost:8000',
      changeOrigin: true
    }
  }
}

```

Esto permite que el SPA llame a `/api/stores/` y Vite lo redirige a `http://localhost:8000/api/stores/`, evitando problemas de CORS en desarrollo.

6.3 Cliente HTTP — Axios

`src/api/axiosConfig.js` crea una instancia Axios con:

- `baseUrl`: `VITE_API_URL` o `http://localhost:8000`
- `timeout`: `10 000 ms`
- **Request interceptor**: inyecta `Authorization: Bearer <token>` desde `localStorage('ss_access')`
- **Response interceptor**: normaliza errores en un único mensaje string


```
api.interceptors.request.use((config) => {
  const token = localStorage.getItem('ss_access')
  if (token) config.headers.Authorization = `Bearer ${token}`
  return config
})
```

6.4 Context API

AUTHCONTEXT

user	object \ null	Datos del usuario; null si no autenticado
loading	boolean	true mientras valida el token inicial
login(email, pw)	async fn	Llama a /api/auth/login/, persiste tokens
logout()	async fn	Blacklistea refresh, limpia localStorage
register(formData)	async fn	Registra y autentica en una sola llamada
updateProfile(data)	async fn	PUT /api/auth/profile/, actualiza user

Inicialización: Al montar, lee `ss_access` de `localStorage`. Si existe, llama a `/api/auth/me/` para reconstituir `user` sin requerir re-login. Si la llamada falla, limpia los tokens.

CARTCONTEXT

items	array	Lista de {product_id, name, price, image, quantity, store_name}
count	number	Total de unidades en el carrito
subtotal	number	Suma de price * quantity
addToCart(product, qty)	fn	Añade o acumula cantidad
removeFromCart(id)	fn	Elimina artículo completamente
updateQuantity(id, qty)	fn	Actualiza cantidad; elimina si qty ≤ 0
clearCart()	fn	Vacía el carrito

Persistencia: El estado se sincroniza con `localStorage('ss_cart')` mediante `useEffect`. Se rehidrata al montar.

TOASTCONTEXT

```
toast('Mensaje de éxito', { type: 'success' }) // type: success|error|info|warning
```

6.5 Rutas (React Router v6)

/	→ <StoresCatalog />	público
/login	→ <LoginPage />	público (redirect si autenticado)
/registro	→ <RegisterPage />	público (redirect si autenticado)
/tienda/:slug	→ <StorePage />	público
/producto/:slug	→ <ProductDetailPage />	público
/carrito	→ <CartPage />	público
/checkout	→ <CheckoutPage />	público (guarda vacío redirect)
/perfil	→ <ProfilePage />	protegido (redirect a /login)

6.6 Patrones de componentes

Componente con fallback de imagen:

```
const [imgError, setImgError] = useState(false)
const showImage = product.image && !imgError

{showImage
  ? <img src={product.image} onError={() => setImgError(true)} />
  : <div className="fallback"><ShoppingBag /></div>
}
```

Skeleton loading:

```
// ProductCardSkeleton usa clases Tailwind 'shimmer'
// definidas en el CSS global con animación de gradiente
<div className="h-4 w-full shimmer rounded" />
```

Composición de clases:

```
import { cn } from '../lib/utils'
// cn() = clsx() + twMerge() — resuelve conflictos Tailwind
<button className={cn('base-class', condition && 'conditional-class')} />
```

6.7 TanStack Query — Estrategia de cache

['stores']	default (0)	Lista de tiendas
['store', slug]	default	Detalle de tienda
['store-products', slug, params]	default	Productos filtrados
['ontology-filters', class]	Infinity	Filtros OWL (estáticos)

Las queries se refetching automáticamente al refocus de ventana (comportamiento por defecto de TanStack Query v5).

CAPÍTULO 7 — BACKEND — ARQUITECTURA DJANGO

7.1 Estructura de apps

```
backend/
├─ manage.py
├─ requirements.txt
├─ .env.example
├─ db.sqlite3          ← Solo en desarrollo (USE_SQLITE=True)
├─ ontology/
│  └─ smartstore.owl   ← Ontología OWL 2.0 en RDF/XML
├─ config/
│  ├─ settings.py      ← Configuración central
│  ├─ urls.py          ← URL root: admin, api/auth, api/stores, etc.
│  ├─ wsgi.py          ← WSGI application (Gunicorn)
│  └─ asgi.py          ← ASGI application (Daphne/Uvicorn)
└─ apps/
   ├─ users/           ← Autenticación, registro, perfil
   ├─ stores/          ← Modelo Store, ViewSet, fixtures
   ├─ products/        ← Modelo Product, ViewSet, filtros
   ├─ orders/          ← Modelo Order/OrderItem, serializers con FSM
   └─ ontology/        ← OntologyService, views semánticas
```

7.2 Middleware stack

El orden en `MIDDLEWARE` es relevante; cada capa envuelve a la siguiente:

- | | |
|--|---|
| 1. <code>SecurityMiddleware</code> | ← HTTPS redirect, HSTS headers |
| 2. <code>CorsMiddleware</code> | ← Cabeceras CORS (debe ir antes de <code>SessionMiddleware</code>) |
| 3. <code>SessionMiddleware</code> | ← Gestión de sesiones |
| 4. <code>CommonMiddleware</code> | ← URL normalization, ETags |
| 5. <code>CsrfViewMiddleware</code> | ← Protección CSRF (DRF usa JWT, no CSRF) |
| 6. <code>AuthenticationMiddleware</code> | ← Autenticación de sesión Django |
| 7. <code>MessageMiddleware</code> | ← Django messages framework |
| 8. <code>XFrameOptionsMiddleware</code> | ← X-Frame-Options: DENY |

7.3 Configuración DRF

```
REST_FRAMEWORK = {
    'DEFAULT_AUTHENTICATION_CLASSES': [
        'rest_framework_simplejwt.authentication.JWTAuthentication',
        'rest_framework.authentication.SessionAuthentication',
    ],
    'DEFAULT_PERMISSION_CLASSES': [
        'rest_framework.permissions.IsAuthenticatedOrReadOnly',
    ],
    'DEFAULT_FILTER_BACKENDS': [
        'django_filters.rest_framework.DjangoFilterBackend',
        'rest_framework.filters.SearchFilter',
        'rest_framework.filters.OrderingFilter',
    ],
    'DEFAULT_PAGINATION_CLASS': 'rest_framework.pagination.PageNumberPagination',
    'PAGE_SIZE': 20,
}
```

7.4 Permisos por endpoint

Stores (lista/detalle)	<code>IsAuthenticatedOrReadOnly</code>	Catálogo público
Products (lista/detalle)	<code>IsAuthenticatedOrReadOnly</code>	Catálogo público
Ontology (suggestions, search)	<code>AllowAny</code>	Búsqueda pública
Ontology (reload)	<code>Autenticado + DEBUG=True</code>	Solo desarrollo
Auth (register, login, refresh)	<code>AllowAny</code>	Flujo de autenticación
Auth (me, profile)	<code>IsAuthenticated</code>	Datos personales
Orders	<code>IsAuthenticated</code>	Datos financieros del usuario

7.5 ViewSets y serializers por app

APP STORES

```
StoreViewSet (ListModelMixin + RetrieveModelMixin + GenericViewSet)
├── list()      → StoreListSerializer
├── retrieve()  → StoreDetailSerializer
└── products() → @action GET /api/stores/{slug}/products/
                  ProductListSerializer + ProductFilter + apply_semantic_tag_filters
```

APP PRODUCTS

```
ProductViewSet (ListModelMixin + RetrieveModelMixin + GenericViewSet)
├── list() → ProductListSerializer + filtrado en filter_queryset()
└── retrieve() → ProductDetailSerializer
```

`filter_queryset()` encadena: 1. `DjangoFilterBackend` con `ProductFilter` 2. `SearchFilter` sobre `name`, `description` 3. `OrderingFilter` 4. `apply_semantic_tag_filters()` — filtros dinámicos de `semantic_tags`

APP ORDERS

```
OrderListCreateView (ListCreateAPIView)
├── GET → OrderSerializer (prefetch_related items)
└── POST → OrderSerializer.create() — crea Order + OrderItems en transacción

OrderStatusUpdateView (APIView)
└── PATCH → OrderStatusUpdateSerializer (valida FSM de status)
```

APP USERS

```
RegisterView (APIView, AllowAny)
└── POST → RegisterSerializer + emite JWT

LoginView (TokenObtainPairView)
└── POST → EmailTokenObtainPairSerializer

LogoutView (APIView, AllowAny)
└── POST → RefreshToken(token).blacklist()

MeView (APIView, IsAuthenticated)
└── GET → UserSerializer

ProfileView (APIView, IsAuthenticated)
└── PUT → ProfileUpdateSerializer → UserSerializer
```

7.6 Management commands

`populate_fixtures`

`apps/stores/management/commands/`

Crea 20 tiendas y 300+ productos con imágenes Unsplash

Uso:

```
python manage.py populate_fixtures # inserta/actualiza
python manage.py populate_fixtures --flush # elimina todo y recrea
```

El comando usa `Store.objects.update_or_create(slug=slug, ...)` y `Product.objects.update_or_create(store=store, slug=slug, ...)` para ser idempotente.

CAPÍTULO 8 — ONTOLOGÍA OWL/SPARQL

8.1 Introducción a OWL 2.0

OWL (Web Ontology Language) es un lenguaje de representación de conocimiento basado en RDF desarrollado por el W3C. SmartStore utiliza el perfil **OWL 2 DL** serializado en formato **RDF/XML**.

La ontología define una jerarquía de clases de productos con propiedades, sinónimos y reglas de inferencia que potencian el motor de búsqueda semántica.

8.2 Namespace y prefijos

ss:	http://www.smartstore.org/ontology#	Clases y propiedades SmartStore
owl:	http://www.w3.org/2002/07/owl#	Metavocabulario OWL
rdfs:	http://www.w3.org/2000/01/rdf-schema#	Labels, comments, subClassOf
rdf:	http://www.w3.org/1999/02/22-rdf-syntax-ns#	type, property
xsd:	http://www.w3.org/2001/XMLSchema#	Tipos de datos (string, float)

8.3 Jerarquía de clases (extracto)

```
owl:Thing
├─ ss:Product
│   ├── ss:ElectronicProduct
│   │   ├── ss:Smartphone
│   │   │   ├── ss:AndroidSmartphone
│   │   │   └─ ss:iOSSmartphone
│   │   ├── ss:Laptop
│   │   ├── ss:Tablet
│   │   └─ ss:AudioProduct
│   │       ├── ss:Headphone
│   │       └─ ss:Speaker
│   ├── ss:SportswearProduct
│   │   ├── ss:RunningShoe
│   │   ├── ss:TrainingShoe
│   │   └─ ss:Apparel
│   ├── ss:FurnitureProduct
│   │   ├── ss:Sofa
│   │   └─ ss:Desk
│   ├── ss:BeautyProduct
│   │   ├── ss:Skincare
│   │   └─ ss:Fragrance
│   └─ ss:FoodProduct
│       ├── ss:Beverage
│       └─ ss:Snack
```

8.4 Propiedades personalizadas SmartStore

ss:synonym	owl:AnnotationProperty	owl:Class	Términos alternativos para búsqueda
ss:semanticWeight	owl:DatatypeProperty	owl:Class	Peso de scoring (float, default 1.0)
ss:isFilterable	owl:DatatypeProperty	owl:ObjectProperty	"true" si la propiedad debe exponerse en el panel
ss:filterType	owl:DatatypeProperty	owl:ObjectProperty	"select", "range", "boolean"
ss:filterValues	owl:DatatypeProperty	owl:ObjectProperty	JSON array de valores permitidos
ss:compatibleWith	owl:ObjectProperty	owl:Class	Clases compatibles para tags de compatibilidad
ss:compatibilityConfidence	owl:DatatypeProperty	owl:Class	Confianza de compatibilidad (float)

CLASE INFERENCERULE

```

ss:InferenceRule
  ss:condition      → string (expr evaluable: "rating >= 4.5", "has_discount")
  ss:tag            → string (etiqueta generada: "Producto Premium")
  ss:confidence     → float  (confianza del tag: 0.0 - 1.0)
  ss:targetClass    → URI    (clase objetivo; si ausente, aplica a todos)

```

8.5 Ejemplo de fragmento OWL (RDF/XML)

```

<owl:Class rdf:about="http://www.smartstore.org/ontology#Smartphone">
  <rdfs:subClassOf rdf:resource="http://www.smartstore.org/ontology#ElectronicProduct"/>
  <rdfs:label xml:lang="es">Smartphone</rdfs:label>
  <rdfs:comment xml:lang="es">Teléfonos inteligentes con sistema operativo móvil</rdfs:comment>
  <ss:synonym xml:lang="es">teléfono inteligente</ss:synonym>
  <ss:synonym xml:lang="es">móvil</ss:synonym>
  <ss:synonym xml:lang="es">celular</ss:synonym>
  <ss:semanticWeight rdf:datatype="xsd:float">1.5</ss:semanticWeight>
</owl:Class>

<owl:ObjectProperty rdf:about="http://www.smartstore.org/ontology#hasBrand">
  <rdfs:domain rdf:resource="http://www.smartstore.org/ontology#ElectronicProduct"/>
  <rdfs:label xml:lang="es">Marca</rdfs:label>
  <ss:isFilterable>true</ss:isFilterable>
  <ss:filterType>select</ss:filterType>
  <ss:filterValues>["Samsung","Apple","Xiaomi","OnePlus","Google"]</ss:filterValues>
</owl:ObjectProperty>

<ss:InferenceRule rdf:about="http://www.smartstore.org/ontology#Rule_HighRated">
  <ss:condition>rating >= 4.5</ss:condition>
  <ss:tag>Muy valorado</ss:tag>
  <ss:confidence rdf:datatype="xsd:float">0.95</ss:confidence>
</ss:InferenceRule>

```

8.6 Patrón singleton del grafo

```

# apps/ontology/services.py
_graph: Graph | None = None

def _get_graph() -> Graph:
    global _graph
    if _graph is None:
        g = Graph()
        g.parse(str(settings.ONTOLGY_PATH), format='xml')
        _graph = g
    return _graph

```

El grafo se carga una única vez en memoria al primer uso. Se puede recargar en desarrollo mediante `POST /api/ontology/reload/` o llamando a `reload_graph()`.

☐ **Advertencia de concurrencia:** En producción con múltiples workers Unicorn, cada proceso cargará su propia copia del grafo. Esto es aceptable pero implica uso de memoria multiplicado por el número de workers. Con 4 workers y un grafo de 50MB, se usarán ~200MB solo para la ontología.

8.7 Cómo extender la ontología

1. Editar `backend/ontology/smartstore.owl` con un editor OWL (ej. Protégé).
2. Añadir la nueva clase como subclase de la clase padre apropiada.
3. Definir `rdfs:label` en español y los `ss:synonym` correspondientes.
4. Añadir propiedades filtrables con `ss:isFilterable = "true"`.
5. En desarrollo, llamar a `POST /api/ontology/reload/` para recargar sin reiniciar.

6. En producción, reiniciar los workers de Gunicorn: `sudo systemctl restart gunicorn`.

CAPÍTULO 9 — BÚSQUEDA SEMÁNTICA

9.1 Flujo completo (diagrama)



```
OUTPUT
{
  products: [42, 17, 88, ...], ← IDs
  inferred_class: "RunningShoe",
  confidence: 1.0,
  matched_tokens: ["zapatillas", "running", "ligeras"],
  expanded_classes: ["RunningShoe", "TrailRunningShoe"]
}
```

9.2 Fallback de texto plano

Si `class_scores` está vacío (ningún token coincide con la ontología), se ejecuta `_fallback_text_search()` :

```
q = Q()
for token in tokens:
    q |= Q(name__icontains=token) | Q(description__icontains=token)
```

Retorna `confidence: 0.0` e `inferred_class: null`.

9.3 Condiciones de InferenceRule

<code>has_discount</code>	<code>has_discount</code>	<code>compare_price > price</code>
<code>has_tag:<valor></code>	<code>has_tag:5G</code>	<code>"5G" in semantic_tags</code>
<code>campo op valor</code>	<code>rating >= 4.5</code>	Comparación numérica del campo del modelo
<code>campo op string</code>	<code>ontology_class == .../Smartphone</code>	Comparación de string

9.4 Limitaciones actuales

Idioma único	Los labels y sinónimos son solo en español
Sin lematización	“correr” no coincide con “corriendo”
Sin embeddings	No usa vectores semánticos; depende de coincidencias léxicas
Grafo en memoria	No escala a ontologías de más de ~100K triples sin caché externa
50 resultados máx	<code>product_ids[:50]</code> hardcoded en <code>semantic_search()</code>
Sin ranking interno	Los productos del fallback se ordenan por el ORM, no por relevancia

CAPÍTULO 10 — SEGURIDAD

10.1 Autenticación JWT

Algoritmo	HS256 (HMAC-SHA256, simétrico)
Access token TTL	60 minutos

Refresh token TTL	7 días
Rotación	<code>ROTATE_REFRESH_TOKENS = True</code> — nuevo refresh en cada renovación
Blacklist	<code>BLACKLIST_AFTER_ROTATION = True</code> — invalida refresh anterior
Almacenamiento cliente	<code>localStorage</code> (keys: <code>ss_access</code> , <code>ss_refresh</code>)
Cabecera	<code>Authorization: Bearer <token></code>

□ **localStorage vs httpOnly cookies:** El uso de `localStorage` expone los tokens a JavaScript. En producción con alta sensibilidad de seguridad, se recomienda migrar a `httpOnly` cookies con `SameSite=Strict` para mitigar XSS.

10.2 Protección contra vulnerabilidades OWASP Top 10

A01 Broken Access Control	<code>IsAuthenticated</code> en endpoints de órdenes y perfil; usuario solo ve sus propias órdenes
A02 Cryptographic Failures	Contraseñas con PBKDF2-SHA256 (Django default); JWT con HS256
A03 Injection	Django ORM con prepared statements; no hay SQL raw
A04 Insecure Design	FSM de status de órdenes previene transiciones ilegales
A05 Security Misconfiguration	<code>SECRET_KEY</code> en <code>.env</code> , <code>DEBUG=False</code> en producción, <code>ALLOWED_HOSTS</code> configurado
A07 Identity Failures	Validadores de contraseña Django (longitud mínima, similitud, común)
A09 Logging Failures	Django logging configurado; errores en <code>semantic_search</code> logeados

10.3 CORS

```
CORS_ALLOWED_ORIGINS = ['http://localhost:5173', 'http://127.0.0.1:5173']
```

Solo el origen del SPA está permitido. En producción se debe restringir al dominio de producción del frontend.

10.4 CSRF

DRF con JWT no utiliza sesiones para la API REST, por lo que las peticiones a `/api/` no requieren token CSRF. El `CsrfViewMiddleware` protege la interfaz de admin de Django (`/admin/`).

10.5 Variables de entorno sensibles

Las variables en `.env` (gestionadas con `python-decouple`) que **nunca** deben committearse al repositorio:

```
SECRET_KEY          ← clave Django; cambiarla invalida todas las sesiones
DB_PASSWORD         ← contraseña de PostgreSQL
DB_USER / DB_NAME   ← credenciales de base de datos
```

10.6 Validación de entrada

- **Serializers DRF:** Validación de tipos, longitudes y formatos en cada campo.
- **Contraseñas:** Validadores de Django en `AUTH_PASSWORD_VALIDATORS`.
- **Email:** Regex en `RegisterSerializer` y `EmailTokenObtainPairSerializer`.
- **Precio:** `MinValueValidator(0)` en campos `price` y `compare_price`.
- **Rating:** `MinValueValidator(0)` + `MaxValueValidator(5)`.
- **SPARQL injection:** Los tokens de búsqueda se sanitizan con `replace("'", '\\')` antes de interpolarse en queries SPARQL.

CAPÍTULO 11 — RENDIMIENTO Y OPTIMIZACIÓN

11.1 Optimizaciones de base de datos

<code>select_related('store')</code>	<code>ProductViewSet.queryset</code>	Evita N+1 en productos
<code>prefetch_related('items')</code>	<code>OrderListCreateView.get_queryset()</code>	Evita N+1 en historial
Índices compuestos	<code>Product.Meta.indexes</code>	Filtros de precio, rating y ontology
Índice <code>is_active</code>	<code>Store, Product</code>	Filtro más frecuente del catálogo
<code>pagination_class = None</code>	<code>OrderListCreateView</code>	Evita <code>COUNT(*)</code> en historial pequeño

11.2 Índices por tabla

<code>stores_store</code>	<code>name</code>	B-Tree
<code>stores_store</code>	<code>(is_active, name)</code>	B-Tree compuesto
<code>products_product</code>	<code>(store_id, is_active)</code>	B-Tree compuesto
<code>products_product</code>	<code>price</code>	B-Tree
<code>products_product</code>	<code>rating</code>	B-Tree
<code>products_product</code>	<code>(store_id, price)</code>	B-Tree compuesto
<code>products_product</code>	<code>(ontology_class, is_active)</code>	B-Tree compuesto

11.3 Caché del grafo ontológico

El grafo rdflib se carga como singleton de módulo (`_graph: Graph | None = None`). Esto significa:

- **Primera petición:** ~0.5–2 segundos de carga del OWL.
- **Peticiones subsiguientes:** Microsegundos (búsqueda en memoria).
- **En producción:** Cada worker Gunicorn tiene su propia copia. Con `--workers 4` se cargarán 4 copias.

Recomendación para escala: Usar un proceso de pre-carga o reducir workers y aumentar threads si la RAM es limitada.

11.4 Optimizaciones del frontend

Tree shaking	Vite + ES modules; solo se incluyen los íconos Lucide importados
Code splitting	React Router v6 usa lazy loading automático por ruta
Server state cache	TanStack Query cachea respuestas y evita refetches innecesarios
Skeleton loading	<code>LoadingGrid.jsx</code> muestra skeletons mientras carga para UX fluida
Image fallback	<code>onError</code> + estado local evita peticiones rotas colgadas
Debounce en búsqueda	El Header aplica debounce (~300ms) antes de llamar a <code>/suggestions/</code>

11.5 Imágenes (CDN Unsplash)

Todas las imágenes de productos y tiendas usan URLs de **Unsplash CDN** con parámetros de optimización:

```
?auto=format&fit=crop&w=600&q=80    ← productos
?auto=format&fit=crop&w=200&h=200    ← logos
?auto=format&fit=crop&w=1200&h=400   ← banners
```

auto=format sirve WebP en navegadores compatibles; q=80 reduce peso ~40% vs calidad máxima.

CAPÍTULO 12 — DEPLOYMENT

12.1 Requisitos de servidor

CPU	1 vCPU	2 vCPU
RAM	2 GB	4 GB
Disco	20 GB	50 GB SSD
SO	Ubuntu 22.04 LTS	Ubuntu 24.04 LTS
Python	3.11	3.12
Node.js	18	20 LTS
PostgreSQL	14	16

12.2 Instalación del backend en producción

```
# 1. Crear usuario del sistema
sudo useradd -m -s /bin/bash smartstore
sudo su - smartstore

# 2. Clonar repositorio
git clone <repo-url> /home/smartstore/app
cd /home/smartstore/app/backend

# 3. Entorno virtual
python3.12 -m venv venv
source venv/bin/activate
pip install -r requirements.txt
pip install gunicorn # servidor WSGI

# 4. Variables de entorno
cp .env.example .env
nano .env # configurar SECRET_KEY, DB_*, etc.

# 5. Migraciones y datos
python manage.py migrate
python manage.py collectstatic --noinput
python manage.py populate_fixtures

# 6. Iniciar Gunicorn (prueba)
gunicorn config.wsgi:application --bind 0.0.0.0:8000 --workers 4
```

12.3 Gunicorn como servicio systemd

```
# /etc/systemd/system/smartstore.service
[Unit]
Description=SmartStore Gunicorn
After=network.target

[Service]
User=smartstore
WorkingDirectory=/home/smartstore/app/backend
ExecStart=/home/smartstore/app/backend/venv/bin/gunicorn \
    config.wsgi:application \
    --workers 4 \
    --bind unix:/run/smartstore.sock \
    --timeout 60 \
    --access-logfile /var/log/smartstore/access.log \
    --error-logfile /var/log/smartstore/error.log
Restart=on-failure

[Install]
WantedBy=multi-user.target
```

```
sudo systemctl enable smartstore
sudo systemctl start smartstore
```

12.4 Nginx como reverse proxy

```
# /etc/nginx/sites-available/smartstore
server {
    listen 443 ssl;
    server_name api.smartstore.example.com;

    ssl_certificate      /etc/letsencrypt/live/api.smartstore.example.com/fullchain.pem;
    ssl_certificate_key  /etc/letsencrypt/live/api.smartstore.example.com/privkey.pem;
    ssl_protocols       TLSv1.2 TLSv1.3;

    location / {
        proxy_pass http://unix:/run/smartstore.sock;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }

    location /static/ {
        alias /home/smartstore/app/backend/staticfiles/;
        expires 1y;
        add_header Cache-Control "public, immutable";
    }

    location /media/ {
        alias /home/smartstore/app/backend/media/;
    }
}
```

12.5 Build y despliegue del frontend

```
cd frontend

# Variables de entorno de producción
echo "VITE_API_URL=https://api.smartstore.example.com" > .env.production

# Build
npm run build
# Genera dist/ con archivos estáticos

# Subir a S3 (o servidor de archivos estáticos)
aws s3 sync dist/ s3://smartstore-frontend/ --delete
aws cloudfront create-invalidation --distribution-id <ID> --paths "/*"
```

12.6 Variables de entorno de producción

```
# backend/.env (producción)
SECRET_KEY=<clave-aleatoria-256-bits>
DEBUG=False
ALLOWED_HOSTS=api.smartstore.example.com
USE_SQLITE=False
DB_NAME=smartstore_db
DB_USER=smartstore_user
DB_PASSWORD=<contraseña-segura>
DB_HOST=localhost
DB_PORT=5432
CORS_ALLOWED_ORIGINS=https://www.smartstore.example.com
```

12.7 Settings de producción adicionales

En `settings.py` agregar para producción:

```
# Forzar HTTPS
SECURE_SSL_REDIRECT = True
SECURE_HSTS_SECONDS = 31536000
SECURE_HSTS_INCLUDE_SUBDOMAINS = True
SECURE_HSTS_PRELOAD = True

# Cookies seguras
SESSION_COOKIE_SECURE = True
CSRF_COOKIE_SECURE = True

# Logging
LOGGING = {
    'version': 1,
    'handlers': {
        'file': {
            'class': 'logging.FileHandler',
            'filename': '/var/log/smartstore/django.log',
        },
    },
    'root': {
        'handlers': ['file'],
        'level': 'WARNING',
    },
}
```

CAPÍTULO 13 — TESTING

13.1 Estrategia de testing

SmartStore sigue la pirámide de testing: mayor cantidad de tests unitarios, menor cantidad de tests E2E.

```

/\
/E2E\      ← Playwright/Cypress (no implementados)
/-----\
/ Integración\ ← DRF APITestCase (implementado parcialmente)
/-----\
/   Unitarios   \ ← pytest + unittest (implementados)
/-----\

```

13.2 Tests de backend

Ejecutar tests:

```

cd backend
python manage.py test apps.orders.tests apps.users.tests

```

Estructura de tests existentes:

```

apps/
├── orders/
│   ├── tests.py ← OrderModel, OrderStatusFSM, OrderSerializer
├── users/
│   ├── tests.py ← RegisterView, LoginView, ProfileView

```

Ejemplo — Test de transición de estado:

```

class OrderFSMTest(TestCase):
    def test_pending_to_completed(self):
        order = Order.objects.create(user=self.user, status='pending', ...)
        data = {'status': 'completed'}
        serializer = OrderStatusUpdateSerializer(order, data=data, partial=True)
        self.assertTrue(serializer.is_valid())
        serializer.save()
        order.refresh_from_db()
        self.assertEqual(order.status, 'completed')

    def test_completed_cannot_transition(self):
        order = Order.objects.create(user=self.user, status='completed', ...)
        serializer = OrderStatusUpdateSerializer(order, data={'status': 'pending'}, partial=True)
        self.assertFalse(serializer.is_valid())

```

13.3 Tests de API (Integration)

```

from rest_framework.test import APITestCase
from rest_framework import status

class StoreAPITest(APITestCase):
    def test_list_stores(self):
        response = self.client.get('/api/stores/')
        self.assertEqual(response.status_code, status.HTTP_200_OK)
        self.assertIsInstance(response.data, list)

    def test_store_products_require_valid_slug(self):
        response = self.client.get('/api/stores/no-existe/products/')
        self.assertEqual(response.status_code, status.HTTP_404_NOT_FOUND)

```

13.4 Cobertura de tests

```

pip install coverage
coverage run --source='.' manage.py test
coverage report
coverage html # genera htmlcov/index.html

```

13.5 Tests del motor ontológico

```

class OntologyServiceTest(TestCase):
    def setUp(self):
        self.service = OntologyService()

    def test_tokenize_removes_stopwords(self):
        tokens = self.service._tokenize("zapatillas de running")
        self.assertNotIn("de", tokens)
        self.assertIn("zapatillas", tokens)

    def test_semantic_search_returns_dict(self):
        result = self.service.semantic_search("smartphone")
        self.assertIn("products", result)
        self.assertIn("confidence", result)
        self.assertIsInstance(result["confidence"], float)

```

CAPÍTULO 14 — TROUBLESHOOTING TÉCNICO

14.1 Tabla de errores comunes

ImproperlyConfigured: SECRET_KEY	Variable no configurada	Definir SECRET_KEY en .env
OperationalError: no such table	Migraciones no ejecutadas	python manage.py migrate
ConnectionRefusedError: [Errno 111]	PostgreSQL no activo	sudo systemctl start postgresql
FileNotFoundError: smartstore.owl	ONTOLOGY_PATH incorrecto	Verificar backend/ontology/smartstore.owl existe
CORS: blocked by policy	Origen no en CORS_ALLOWED_ORIGINS	Añadir el origen al setting
401 Unauthorized en API	Token expirado o ausente	Llamar a /api/auth/token/refresh/
400 Cannot transition from...	Transición de status inválida	Respetar el FSM de órdenes
json_each no encontrado	SQLite < 3.38	Actualizar SQLite o usar PostgreSQL
ModuleNotFoundError: rdflib	Entorno no activado	source venv/bin/activate
Imágenes con URL localhost/media/https://	Campo ImageField serializa URLs externas	Usar URLField (ya corregido en migración 0002)

14.2 Debugging del motor semántico

```
# Shell interactivo Django
python manage.py shell

# Probar tokenización
from apps.ontology.services import OntologyService
svc = OntologyService()
print(svc._tokenize("zapatillas de running ligeras"))

# Probar búsqueda
result = svc.semantic_search("smartphone barato")
print(result['inferred_class'], result['confidence'])

# Ver triples del grafo
g = svc.graph
print(f"Total triples: {len(g)}")

# Listar clases
for cls in g.subjects(RDF.type, OWL.Class):
    print(cls)
```

14.3 Logs importantes

Django	Consola (runserver)	Peticiones HTTP, errores, warnings
Ontología	Logger apps.ontology.services	Carga del grafo, número de triples
Gunicorn	/var/log/smartstore/access.log	Peticiones HTTP con tiempos
Gunicorn errors	/var/log/smartstore/error.log	Excepciones no controladas

14.4 Performance debugging

```
# Identificar N+1 queries con django-debug-toolbar (desarrollo)
pip install django-debug-toolbar

# O con logging de SQL
import logging
logging.getLogger('django.db.backends').setLevel(logging.DEBUG)
```

```
# Verificar índices en SQLite
python manage.py dbshell
.schema products_product
EXPLAIN QUERY PLAN SELECT * FROM products_product WHERE is_active=1 AND price < 1000;
```

CAPÍTULO 15 — GLOSARIO TÉCNICO

ALLOWED_HOSTS	Lista de dominios permitidos en Django; previene ataques de Host header
APIView	Clase base de DRF para vistas que no usan ViewSets; control total del dispatch
auto_now / auto_now_add	Parámetros de DateTimeField; <code>auto_now_add</code> registra creación, <code>auto_now</code> actualización
Axios interceptor	Función que transforma peticiones o respuestas globalmente antes de que lleguen al código de la app

Bearer token	Esquema de autenticación HTTP donde el token se incluye literalmente en la cabecera Authorization
Blacklist	Lista de tokens JWT invalidados antes de su expiración natural; implementada con <code>rest_framework_simplejwt.token_blacklist</code>
CorsMiddleware	Middleware que añade cabeceras HTTP CORS a las respuestas, controlando qué orígenes pueden acceder a la API
db_index	Parámetro de campo Django que crea un índice B-Tree en la columna correspondiente
DEFAULT_AUTO_FIELD	Tipo de campo PK automático por defecto; <code>BigAutoField</code> usa <code>BIGINT</code> (64 bits)
DjangoFilterBackend	Backend de filtrado que integra <code>django-filter</code> con DRF mediante <code>FilterSet</code> declarativos
FSM	Finite State Machine; máquina de estados finitos que modela transiciones válidas de estado
GenericViewSet	ViewSet DRF sin mixins; se combinan <code>ListModelMixin</code> , <code>RetrieveModelMixin</code> , etc. según necesidad
grafo RDF	Conjunto de triples (<code>sujeto</code> , <code>predicado</code> , <code>objeto</code>) que representan conocimiento estructurado
HMR	Hot Module Replacement; Vite actualiza módulos individuales sin recargar la página completa
IRI	Internationalized Resource Identifier; generalización de URI que permite caracteres Unicode
isFilterable	Propiedad custom OWL (<code>ss:isFilterable</code>) que marca una propiedad para exposición en el panel de filtros
json_each()	Función SQLite que itera los elementos de un array JSON; usada para filtrar <code>semantic_tags</code>
JSONField	Campo Django que almacena JSON nativo (PostgreSQL <code>jsonb</code> , SQLite <code>text</code>)
localStorage	API del navegador para persistencia de datos por origen; SmartStore guarda tokens JWT y carrito
lualatex / xelatex	Motores LaTeX con soporte Unicode; alternativas a pdflatex para documentos multilingües
MinValueValidator	Validador DRF/Django que restringe el valor mínimo de un campo numérico
Namespace (OWL)	Prefijo de IRI que agrupa clases y propiedades de una ontología; en SmartStore es <code>http://www.smartstore.org/ontology#</code>
ORM	Object-Relational Mapper; abstracción que traduce objetos Python a SQL; Django ORM usa <code>QuerySets</code>
owl:Class	Metaclass OWL que define una categoría de individuos; base de la jerarquía de productos
owl:DatatypeProperty	Propiedad OWL que relaciona un individuo con un valor de tipo de dato (string, float, etc.)
owl:ObjectProperty	Propiedad OWL que relaciona dos individuos dentro de la ontología
PageNumberPagination	Clase DRF que divide resultados en páginas accesibles via <code>?page=N</code>
PBKDF2-SHA256	Algoritmo de derivación de clave; Django lo usa para hashear contraseñas (100,000+ iteraciones)

prefetch_related	Método de QuerySet que pre-carga relaciones inversas con una segunda query; evita N+1
proxy (Vite)	Redirección de peticiones <code>/api/*</code> al servidor backend; evita CORS en desarrollo
RDFS	RDF Schema; vocabulario de metadatos para definir jerarquías de clases y propiedades
rdflib	Biblioteca Python para trabajar con grafos RDF; soporta parsing OWL/XML y consultas SPARQL
rdfs:subClassOf	Propiedad RDFS que define herencia entre clases; <code>Smartphone rdfs:subClassOf ElectronicProduct</code>
RefreshToken	Token de larga duración (7 días) usado para obtener nuevos access tokens sin re-autenticación
related_name	Parámetro de FK/OneToOne que define el nombre del accessor inverso en Python
SELECT_RELATED	Método QuerySet que resuelve FKs con SQL JOIN; reduce queries para relaciones directas
serializer.is_valid()	Método DRF que ejecuta validación de datos; retorna <code>False</code> si hay errores
SET_NULL	Comportamiento de FK al eliminar el objeto referenciado; pone el campo en <code>NULL</code>
singleton	Patrón de diseño que asegura una única instancia de un objeto; el grafo OWL usa este patrón
slug	Identificador legible para URLs; generado con <code>django.utils.text.slugify()</code>
SPARQLWrapper	Biblioteca Python para ejecutar consultas SPARQL sobre endpoints HTTP remotos
ss:synonym	Propiedad de anotación custom que registra términos alternativos para una clase OWL
TanStack Query	Biblioteca de gestión de estado del servidor para React; maneja fetching, caché y sincronización
Token blacklist	Extensión de SimpleJWT que almacena tokens invalidados en la tabla <code>token_blacklist_blacklistedtoken</code>
triple RDF	Unidad atómica de conocimiento: (sujeto, predicado, objeto) ; ej: <code>(Smartphone, subClassOf, ElectronicProduct)</code>
URLField	Campo Django para almacenar URLs; no prepende <code>MEDIA_URL</code> al serializar (a diferencia de ImageField)
uuid4	UUID versión 4 generado aleatoriamente; SmartStore usa <code>uuid4().hex[:8]</code> para números de orden
ViewSet	Clase DRF que agrupa acciones relacionadas (list, retrieve, create...) sobre un mismo recurso
WSGI	Interfaz estándar Python entre servidor web y aplicación; Gunicorn implementa WSGI

APÉNDICE A — ERD COMPLETO

El diagrama ERD completo se encuentra en la sección 4.1 (Capítulo 4). A continuación se complementa con las tablas internas de Django y SimpleJWT relevantes para producción:

```
django_migrations      ← Registro de migraciones aplicadas
django_content_type    ← Tipos de contenido para permisos
auth_permission        ← Permisos del sistema
auth_group             ← Grupos de usuarios
token_blacklist_outstandingtoken ← Tokens JWT emitidos
token_blacklist_blacklistedtoken ← Tokens JWT invalidados (logout)
```

APÉNDICE B — DIAGRAMA DE ARQUITECTURA



APÉNDICE C — QUERIES SPARQL DE REFERENCIA

C.1 Obtener todas las subclases de Product

```
PREFIX owl: <http://www.w3.org/2002/07/owl#>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
PREFIX ss: <http://www.smartstore.org/ontology#>

SELECT DISTINCT ?class ?label ?parent WHERE {
  ?class a owl:Class .
  ?class rdfs:subClassOf+ ss:Product .
  OPTIONAL { ?class rdfs:label ?label FILTER(LANG(?label) = "es") }
  OPTIONAL { ?class rdfs:subClassOf ?parent FILTER(?parent != owl:Thing) }
}

ORDER BY ?label
```

C.2 Buscar clase por label o sinónimo

```
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
PREFIX owl: <http://www.w3.org/2002/07/owl#>
PREFIX ss: <http://www.smartstore.org/ontology#>

SELECT ?class ?weight WHERE {
  ?class a owl:Class .
  {
    ?class rdfs:label ?lbl
    FILTER(CONTAINS(LCASE(STR(?lbl)), LCASE("smartphone")))
  } UNION {
    ?class ss:synonym ?syn
    FILTER(CONTAINS(LCASE(STR(?syn)), LCASE("smartphone")))
  }
  OPTIONAL { ?class ss:semanticWeight ?weight }
}
```

C.3 Filtros filtrables de una clase y sus ancestros

```
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
PREFIX ss: <http://www.smartstore.org/ontology#>

SELECT DISTINCT ?prop ?label ?filterType ?filterValues WHERE {
  ?prop rdfs:domain <http://www.smartstore.org/ontology#Smartphone> ;
  ss:isFilterable "true" .
  OPTIONAL { ?prop rdfs:label ?label FILTER(LANG(?label) = "es") }
  OPTIONAL { ?prop ss:filterType ?filterType }
  OPTIONAL { ?prop ss:filterValues ?filterValues }
}
```

C.4 Obtener todas las InferenceRules

```
PREFIX ss: <http://www.smartstore.org/ontology#>

SELECT ?condition ?tag ?confidence ?targetClass WHERE {
  ?rule a ss:InferenceRule ;
  ss:condition ?condition ;
  ss:tag ?tag .
  OPTIONAL { ?rule ss:confidence ?confidence }
  OPTIONAL { ?rule ss:targetClass ?targetClass }
}

ORDER BY DESC(?confidence)
```

C.5 Relaciones de compatibilidad

```
PREFIX ss: <http://www.smartstore.org/ontology#>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>

SELECT ?targetLabel ?confidence WHERE {
  <http://www.smartstore.org/ontology#Headphone> ss:compatibleWith ?target .
  ?target rdfs:label ?targetLabel FILTER(LANG(?targetLabel) = "es") .
  OPTIONAL { <http://www.smartstore.org/ontology#Headphone>
    ss:compatibilityConfidence ?confidence }
}
```

APÉNDICE D — VARIABLES DE ENTORNO

```
# backend/.env.example

# — Django Core —
SECRET_KEY=django-insecure-CHANGE-ME-IN-PRODUCTION-USE-RANDOM-256-BIT-KEY
DEBUG=True
ALLOWED_HOSTS=localhost,127.0.0.1

# — Base de datos —
USE_SQLITE=True           # True=SQLite (desarrollo), False=PostgreSQL (producción)
DB_NAME=smartstore_db
DB_USER=postgres
DB_PASSWORD=
DB_HOST=localhost
DB_PORT=5432

# — CORS —
CORS_ALLOWED_ORIGINS=http://localhost:5173,http://127.0.0.1:5173

# — Notas de producción —
# SECRET_KEY: generar con:
#   python -c "from django.core.management.utils import get_random_secret_key; print(get_random_secret_key())"
# DEBUG=False en producción (NUNCA True en servidor público)
# ALLOWED_HOSTS debe incluir el dominio real del servidor
# CORS_ALLOWED_ORIGINS solo el dominio del frontend desplegado
```

APÉNDICE E — SCRIPTS ÚTILES

E.1 Generar SECRET_KEY segura

```
python -c "from django.core.management.utils import get_random_secret_key; print(get_random_secret_key())"
```

E.2 Repoblar base de datos desde cero

```
python manage.py populate_fixtures --flush
```

E.3 Exportar base de datos SQLite a PostgreSQL

```
# 1. Volcar datos
python manage.py dumpdata --natural-foreign --natural-primary \
    --exclude=contenttypes --exclude=auth.permission \
    > dump.json

# 2. Cambiar configuración a PostgreSQL en .env
# USE_SQLITE=False + DB_* configurados

# 3. Crear tablas y cargar
python manage.py migrate
python manage.py loaddata dump.json
```

E.4 Recargar ontología sin reiniciar

```
curl -X POST http://localhost:8000/api/ontology/reload/ \
    -H "Content-Type: application/json"
# Solo funciona con DEBUG=True
```

E.5 Verificar salud del sistema

```
# Backend
curl http://localhost:8000/api/stores/ | python -m json.tool | head -20

# Ontología
curl "http://localhost:8000/api/ontology/classes/" | python -m json.tool | head -20

# JWT
TOKEN=$(curl -s -X POST http://localhost:8000/api/auth/login/ \
    -H "Content-Type: application/json" \
    -d '{"email":"test@example.com","password":"testpass"}' | python -c "import sys,json; print(json.load(sys.stdin))" | python -c "import sys,json; print(json.dumps(json.load(sys.stdin)))"
curl -H "Authorization: Bearer $TOKEN" http://localhost:8000/api/auth/me/
```

E.6 Build de producción del frontend

```
cd frontend
npm run build
# Verificar tamaño del bundle
du -sh dist/
ls -lh dist/assets/
```

APÉNDICE F — REFERENCIAS

- 1 IEEE Std 830-1998. *IEEE Recommended Practice for Software Requirements Specifications*. IEEE, 1998.
- 2 ISO/IEC/IEEE 29148:2018. *Systems and software engineering — Life cycle processes — Requirements engineering*. ISO/IEC/IEEE, 2018.
- 3 ISO/IEC 27001:2022. *Information security, cybersecurity and privacy protection*. ISO/IEC, 2022.
- 4 ISO/IEC 25010:2011. *Systems and software engineering — Quality of software product*. ISO/IEC, 2011.
- 5 W3C. *OWL 2 Web Ontology Language Document Overview (Second Edition)*. W3C Recommendation, 11 December 2012. <https://www.w3.org/TR/owl2-overview/>
- 6 W3C. *SPARQL 1.1 Query Language*. W3C Recommendation, 21 March 2013. <https://www.w3.org/TR/sparql11-query/>

7 W3C. *RDF 1.1 Concepts and Abstract Syntax*. W3C Recommendation, 25 February 2014. <https://www.w3.org/TR/rdf11-concepts/>

8 Jones, M., Bradley, J., Sakimura, N. *JSON Web Token (JWT)*. RFC 7519. IETF, May 2015. <https://www.rfc-editor.org/rfc/rfc7519>

9 Django Software Foundation. *Django 5.1 Documentation*. <https://docs.djangoproject.com/en/5.1/>

10 Django REST Framework. *DRF 3.15 Documentation*. <https://www.django-rest-framework.org/>

11 Facebook Inc. *React 18 Documentation*. <https://react.dev/>

12 Evan You. *Vite Documentation*. <https://vitejs.dev/>

13 Löfgren, I. *rdflib 7.1 Documentation*. <https://rdflib.readthedocs.io/>

14 Hettinga, I. et al. *TanStack Query v5 Documentation*. <https://tanstack.com/query/v5>

15 OWASP. *OWASP Top Ten 2021*. <https://owasp.org/www-project-top-ten/>

CONTROL DEL DOCUMENTO

Documento	SS-MT-001
Revisión	1.0.0
Páginas	Documento digital (Markdown)
Idioma	Español (México)
Normas de referencia	IEEE 830, ISO/IEC/IEEE 29148:2018, ISO/IEC 27001:2022, W3C OWL/SPARQL
Próxima revisión	18 de mayo de 2027

Fin del documento — SmartStore Manual Técnico v1.0.0 · SS-MT-001